

我的核心问题是: 我想要知道`vstate.expect_and_grad(ha)`的具体实现路径是什么. 因为 Netket 这个框架使用了 `plum-dispatch` 这个库, 所以我想要查看源码的举动充满了困难.

完整的代码如下:

```
import jax
import jax.numpy as jnp
import netket as nk
import netket.experimental as nkx
import numpy as np
from pyscf import gto, scf, fci
from flax import linen as nn
import flax.nnx as nnx
import optax
from tqdm import tqdm
from functools import partial

# =====
# 1. 全局参数 & H2 分子定义
# =====
bond_length = 1.4
geometry = [('H', (0., 0., 0.)), ('H', (bond_length, 0., 0.))]
mol = gto.M(atom=geometry, basis='STO-3G', verbose=0)
mf = scf.RHF(mol).run(verbose=0)

cisolver = fci.FCI(mf)
cisolver.nroots = 4
E_fcis, fcivec = cisolver.kernel()
print("="*60)
print("H2 FCI 基准能量")
print("="*60)
for i, e in enumerate(E_fcis):
    exc = (e - E_fcis[0]) * 27.2114
    print(f"E[{i}] = {e:.8f} Ha | 激发能: {exc:.4f} eV")

ha = nkx.operator.from_pyscf_molecule(mol)
hi = nk.hilbert.SpinOrbitalFermions(
    n_orbitals=2,
    s=1/2,
    n_fermions_per_spin=(1,1),
)

# =====
# 2. 神经网络 Ansatz
# =====
class SingleStateAnsatz(nnx.Module):
    def __init__(self, n_spin_orbitals: int, hidden_dim=16, *, rngs: nnx.Rngs):
        super().__init__()
        self.linear1 = nnx.Linear(n_spin_orbitals, hidden_dim, rngs=rngs, param_dtype=complex)
        self.linear2 = nnx.Linear(hidden_dim, hidden_dim, rngs=rngs, param_dtype=complex)
        self.output = nnx.Linear(hidden_dim, 1, rngs=rngs, param_dtype=complex)

    def __call__(self, x):
        h = nnx.tanh(self.linear1(x.astype(complex)))
        h = nnx.tanh(self.linear2(h))
        out = self.output(h)
        return jnp.squeeze(out)

# =====
# 4. 初始化模型、采样器、优化器
# =====
model = SingleStateAnsatz(4, 12, rngs=nnx.Rngs(21))
# 采样器
edges = [(0, 1), (2, 3)]
```

```

g = nk.graph.Graph(edges=edges)
single_rule = nk.sampler.rules.FermionHopRule(hilbert=hi, graph=g)
sampler = nk.sampler.MetropolisSampler(hi, rule=single_rule, n_chains=16, sweep_size=32)

optimizer = nk.optimizer.Sgd(learning_rate=0.1)

vstate = nk.vqs.MCState(sampler, model, n_samples=1008)

gs = nk.driver.VMC(
    ha,
    optimizer,
    variational_state=vstate,
    preconditioner=nk.optimizer.SR(diag_shift=0.1, holomorphic=True),
)
vstate.expect_and_grad(ha)

```

首先由于 `vstate` 是 `MCState` 类型的, 所以 `expect_and_forces` 函数的实现路径是: 在 `MCState/state.py` 中定义:

```

#state.py:
from netket.vqs.base import (
    VariationalState,
    QGTConstructor,
    expect,
    expect_and_grad,
    expect_and_forces,
)

class MCState:
    @timing.timed
    def expect_and_grad(
        self,
        O: AbstractOperator,
        *,
        mutable: CollectionFilter | None = None,
        **kwargs,
    ) -> tuple[Stats, PyTree]:
        if mutable is None:
            mutable = self.mutable

        return expect_and_grad(
            self,
            O,
            self.chunk_size,
            mutable=mutable,
            **kwargs,
        )

```

在 `./vqs/mc/mc_state/expect_grad.py` :

```

@expect_and_grad.dispatch
def expect_and_grad_default_formula(
    vstate: MCState,
    O: AbstractOperator,
    chunk_size: int | None,
    *args,
    mutable: CollectionFilter = False,
    use_covariance: bool | None = None,
) -> tuple[Stats, PyTree]:

    print('调用了./vqs/mc_state/expect_grad.py 下的expect_and_grad_default_formula 函数')
    if use_covariance is None:
        use_covariance = O.is_hermitian

```

```

if use_covariance:
    # Implementation of expect_and_grad for `use_covariance == True` (due to the Literal[True]
    # type in the signature). This case is equivalent to the composition of the
    # `expect_and_forces` and `force_to_grad` functions.
    # return expect_and_grad_from_covariance(vstate,  $\bar{0}$ , *args, mutable=mutable)
     $\bar{0}$ ,  $\bar{0}_{grad}$  = expect_and_forces(vstate,  $\bar{0}$ , chunk_size, *args, mutable=mutable)
     $\bar{0}_{grad}$  = force_to_grad( $\bar{0}_{grad}$ , vstate.parameters)
    return  $\bar{0}$ ,  $\bar{0}_{grad}$ 
else:
    return expect_and_grad_nonhermitian(
        vstate,  $\bar{0}$ , chunk_size, *args, mutable=mutable
    )

```

可以看出是先调用了expect_and_forces函数，再调用了force_to_grad函数。接下来查看这两个函数的定义：

```

@jax.jit
def force_to_grad( $\bar{0}_{grad}$ , parameters):
    """
    Converts the forces vector  $F_k = cov(\bar{0}_k, E_{loc})$  to the observable gradient.
    In case of a complex target (which we assume to correspond to a holomorphic
    parametrization), this is the identity. For real-valued parameters, the gradient
    is 2 Re[F].
    """
     $\bar{0}_{grad}$  = jax.tree_util.tree_map(
        lambda x, target: (x if jnp.iscomplexobj(target) else x.real).astype(
            target.dtype
        ),
         $\bar{0}_{grad}$ ,
        parameters,
    )
     $\bar{0}_{grad}$  = jax.tree_util.tree_map(lambda x: 2 * x,  $\bar{0}_{grad}$ )
    return  $\bar{0}_{grad}$ 

```

```

#
@dispatch
def expect_and_forces( # noqa: F811
    vstate: MCState,
     $\bar{0}$ : AbstractOperator,
    chunk_size: None,
    *,
    mutable: CollectionFilter = False,
) -> tuple[Stats, PyTree]:
     $\sigma$ , args = get_local_kernel_arguments(vstate,  $\bar{0}$ )

    local_estimator_fun = get_local_kernel(vstate,  $\bar{0}$ )

     $\bar{0}$ ,  $\bar{0}_{grad}$ , new_model_state = forces_expect_hermitian(
        local_estimator_fun,
        vstate._apply_fun,
        mutable,
        vstate.parameters,
        vstate.model_state,
         $\sigma$ ,
        args,
    )

    if mutable is not False:
        vstate.model_state = new_model_state

    return  $\bar{0}$ ,  $\bar{0}_{grad}$ 

```

get_local_kernel的具体定义在：vqs/mc/mc_mixed_state/expect.py

```
@dispatch
def get_local_kernel( # noqa: F811
    vstate: MCMixedState, 0: Squared[AbstractSuperOperator]
):
    return kernels.local_value_squared_kernel
```

kernels的定义在: vqs/mc/kernels.py

```
@batch_discrete_kernel
def local_value_kernel(logpsi: Callable, pars: PyTree, σ: Array, args: PyTree):
    """
    local_value kernel for MCState and generic operators
    """
    op, mel = args
    return jnp.sum(mel * jnp.exp(logpsi(pars, op) - logpsi(pars, σ)))

def local_value_squared_kernel(logpsi: Callable, pars: PyTree, σ: Array, args: PyTree):
    """
    local_value kernel for MCState and Squared (generic) operators
    """
    return jnp.abs(local_value_kernel(logpsi, pars, σ, args)) ** 2

def local_value_kernel_jax(
    logpsi: Callable, pars: PyTree, σ: Array, 0: DiscreteJaxOperator
):
    """
    local_value kernel for MCState for jax-compatible operators
    """
    op, mel = 0.get_conn_padded(σ)
    logpsi_σ = logpsi(pars, σ)
    logpsi_op = logpsi(pars, op.reshape(-1, op.shape[-1])).reshape(op.shape[:-1])
    return jnp.sum(mel * jnp.exp(logpsi_op - jnp.expand_dims(logpsi_σ, -1)), axis=-1)
```

forces_expect_hermitian的定义在: vqs/mc/mc_state/expect_forces.py

```
@partial(jax.jit, static_argnums=(0, 1, 2))
def forces_expect_hermitian(
    local_value_kernel: Callable,
    model_apply_fun: Callable,
    mutable: CollectionFilter,
    parameters: PyTree,
    model_state: PyTree,
    σ: jnp.ndarray,
    local_value_args: PyTree,
) -> tuple[PyTree, PyTree]:
    n_chains = σ.shape[0]
    if σ.ndim >= 3:
        σ = jax.lax.collapse(σ, 0, 2)

    n_samples = σ.shape[0]

    O_loc = local_value_kernel(
        model_apply_fun,
        {"params": parameters, **model_state},
        σ,
        local_value_args,
    )

    0̄ = statistics(O_loc.reshape((n_chains, -1)))

    O_loc -= 0̄.mean

    # Then compute the vjp.
    # Code is a bit more complex than a standard one because we support
    # mutable state (if it's there)
```

```

is_mutable = mutable is not False
_, vjp_fun, *new_model_state = nkjax.vjp(
    lambda w: model_apply_fun({"params": w, **model_state}, o, mutable=mutable),
    parameters,
    conjugate=True,
    has_aux=is_mutable,
)
0_grad = vjp_fun(jnp.conjugate(0_loc) / n_samples)[0]

new_model_state = new_model_state[0] if is_mutable else None

return (
    0,
    0_grad,
    new_model_state,
)

```

```

def eval_shape(fun, *args, has_aux=False, **kwargs):
    """
    Returns the dtype of forward_fn(pars, v)
    """
    if has_aux:
        out, _ = jax.eval_shape(fun, *args, **kwargs)
    else:
        out = jax.eval_shape(fun, *args, **kwargs)
    return out

# Copyright 2021 The NetKet Authors - All rights reserved.
#
# Licensed under the Apache License, Version 2.0 (the "License");
# you may not use this file except in compliance with the License.
# You may obtain a copy of the License at
#
#     http://www.apache.org/licenses/LICENSE-2.0
#
# Unless required by applicable law or agreed to in writing, software
# distributed under the License is distributed on an "AS IS" BASIS,
# WITHOUT WARRANTIES OR CONDITIONS OF ANY KIND, either express or implied.
# See the License for the specific language governing permissions and
# limitations under the License.

from typing import Any, Literal, overload, TypeVar
from collections.abc import Callable
from functools import partial

import jax

from jax import numpy as jnp
from jax.tree_util import Partial, tree_map

from netket.utils import HashablePartial

from ._utils_tree import tree_leaf_iscomplex, eval_shape

# These TypeVars are used below to express the fact that function types
# (i.e. call signatures) are invariant under the vmap transformation.
T = TypeVar("T")
U = TypeVar("U")

# _grad_CC, _RR and _RC are the chunked gradient functions for machines going
# from R -> C, R->R and R->C. Ditto for vjp
# The reason why R->C is more complicated is that it splits the calculation
# into the real and complex part in order to be more efficient.

def _cmplx(re, im, conj=False):
    """

```

```

Safely convert real and imaginary part to a complex number, considering
`float0` dtypes which cannot be summed upon.

Those types appear when computing the `vjp` of functions with integer
inputs.
"""
# detect tangent-0 dtypes
is_re_0 = jax.dtypes.issubdtype(re.dtype, jax.dtypes.float0)
is_im_0 = jax.dtypes.issubdtype(re.dtype, jax.dtypes.float0)
if is_re_0 or is_im_0:
    return re
else:
    if conj:
        return re - 1j * im
    else:
        return re + 1j * im

def vjp_fun_cc(out_dtype, conjugate, _vjp_fun, y):
    y = jnp.asarray(y, dtype=out_dtype)

    d_y = _vjp_fun(y)

    if conjugate:
        d_y = tree_map(jnp.conjugate, d_y)

    return d_y

def vjp_cc(
    fun: Callable, *primals, has_aux: bool = False, conjugate: bool = False
) -> tuple[Any, Callable] | tuple[Any, Callable, Any]:
    if has_aux:
        out, _vjp_fun, aux = jax.vjp(fun, *primals, has_aux=True)
    else:
        out, _vjp_fun = jax.vjp(fun, *primals, has_aux=False)

    vjp_fun = Partial(HashablePartial(vjp_fun_cc, out.dtype, conjugate), _vjp_fun)

    if has_aux:
        return out, vjp_fun, aux
    else:
        return out, vjp_fun

def vjp_fun_rr(primals_out_dtype, conjugate, _vjp_fun, y):
    """
    function computing the vjp product for a R->R function.
    """
    if not jnp.iscomplexobj(y):
        out = _vjp_fun(jnp.asarray(y, dtype=primals_out_dtype))
    else:
        out_r = _vjp_fun(jnp.asarray(y.real, dtype=primals_out_dtype))
        out_i = _vjp_fun(jnp.asarray(y.imag, dtype=primals_out_dtype))
        out = tree_map(partial(_cmplx, conj=conjugate), out_r, out_i)

    return out

def vjp_rr(
    fun: Callable, *primals, has_aux: bool = False, conjugate: bool = False
) -> tuple[Any, Callable] | tuple[Any, Callable, Any]:
    if has_aux:
        primals_out, _vjp_fun, aux = jax.vjp(fun, *primals, has_aux=True)
    else:
        primals_out, _vjp_fun = jax.vjp(fun, *primals, has_aux=False)

    vjp_fun = Partial(

```

```

        HashablePartial(vjp_fun_rr, primals_out.dtype, conjugate), _vjp_fun
    )

    if has_aux:
        return primals_out, vjp_fun, aux
    else:
        return primals_out, vjp_fun

def vjp_fun_rc(vals_r_dtype, vals_j_dtype, conjugate, vjp_r_fun, vjp_j_fun, y):
    """
    function computing the vjp product for a R->C function.
    """
    y_r = y.real
    y_j = y.imag

    # val = vals_r + vals_j
    vr_jr = vjp_r_fun(jnp.asarray(y_r, dtype=vals_r_dtype))
    vj_jr = vjp_r_fun(jnp.asarray(y_j, dtype=vals_r_dtype))
    vr_jj = vjp_j_fun(jnp.asarray(y_r, dtype=vals_j_dtype))
    vj_jj = vjp_j_fun(jnp.asarray(y_j, dtype=vals_j_dtype))

    r = tree_map(_cmplx, vr_jr, vj_jr)
    i = tree_map(_cmplx, vr_jj, vj_jj)
    out = tree_map(_cmplx, r, i)

    if conjugate:
        out = tree_map(jnp.conjugate, out)

    return out

def vjp_rc(
    fun: Callable, *primals, has_aux: bool = False, conjugate: bool = False
) -> tuple[Any, Callable] | tuple[Any, Callable, Any]:
    if has_aux:

        def real_fun(*primals):
            val, aux = fun(*primals)
            return val.real, aux

        def imag_fun(*primals):
            val, aux = fun(*primals)
            return val.imag, aux

        vals_r, vjp_r_fun, aux = jax.vjp(real_fun, *primals, has_aux=True)
        vals_j, vjp_j_fun, _ = jax.vjp(imag_fun, *primals, has_aux=True)

    else:
        real_fun = lambda *primals: fun(*primals).real
        imag_fun = lambda *primals: fun(*primals).imag

        vals_r, vjp_r_fun = jax.vjp(real_fun, *primals, has_aux=False)
        vals_j, vjp_j_fun = jax.vjp(imag_fun, *primals, has_aux=False)

    primals_out = vals_r + 1j * vals_j

    vjp_fun = Partial(
        HashablePartial(vjp_fun_rc, vals_r.dtype, vals_j.dtype, conjugate),
        vjp_r_fun,
        vjp_j_fun,
    )

    if has_aux:
        return primals_out, vjp_fun, aux
    else:
        return primals_out, vjp_fun

```

```

# This function dispatches to the right
@overload
def vjp(
    fun: Callable[..., T],
    *primals: Any,
    has_aux: Literal[False] = False,
    conjugate: bool = False,
) -> tuple[T, Callable]: ...

@overload
def vjp(
    fun: Callable[..., tuple[T, U]],
    *primals: Any,
    has_aux: Literal[True] = True, # Fix the default value
    conjugate: bool = False,
) -> tuple[T, Callable, U]: ...

def vjp(
    fun: Callable, *primals, has_aux: bool = False, conjugate: bool = False
) -> tuple[Any, Callable] | tuple[Any, Callable, Any]:
    # output dtype
    out_shape = eval_shape(fun, *primals, has_aux=has_aux)

    if tree_leaf_iscomplex(primals):
        if jnp.iscomplexobj(out_shape): # C -> C
            return vjp_cc(fun, *primals, has_aux=has_aux, conjugate=conjugate)
        else: # C -> R
            return vjp_cc(fun, *primals, has_aux=has_aux, conjugate=conjugate)
    else:
        if jnp.iscomplexobj(out_shape): # R -> C
            return vjp_rc(fun, *primals, has_aux=has_aux, conjugate=conjugate)
        else: # R -> R
            return vjp_rr(fun, *primals, has_aux=has_aux, conjugate=conjugate)

```